

GraphBench Challenge

Réfutation automatique de conjectures en théorie des graphes
Métaheuristiques, IA générative et architecture de type FunSearch

Master 1 MIAGE

Dépôt final : **9 mai à 20h00**

Le projet doit être réalisé en **binôme**. Chaque binôme doit déposer avant le **9 mai à 20h00** un rapport PDF de **5 pages maximum** et fournir un lien vers un dépôt GitHub sur la page de garde du rapport.

1 Contexte scientifique

Une conjecture en théorie des graphes affirme qu'une propriété est vraie pour tous les graphes appartenant à une certaine classe. Ces propriétés relient souvent différents invariants structurels, comme le diamètre, la domination, la connectivité, la taille d'un ensemble indépendant maximum ou encore la taille d'une couverture minimum par sommets.

Une forme générale de conjecture est :

$$\forall G \in \mathcal{C}, \quad A(G) \leq f(B(G)),$$

où :

- G désigne un graphe,
- $\mathcal{C}$ désigne une classe de graphes, par exemple les graphes connexes, planaires, bipartis, les arbres ou les graphes sans griffe,
- $A(G)$ et $B(G)$ sont des invariants du graphe.

Par exemple, dans le benchmark fourni, on rencontre la conjecture suivante portant sur les graphes sans griffe :

$$\forall G \in G_{\text{claw-free}}, \quad \kappa(G) \geq -3 + \kappa'(G)$$

où :

- $\kappa(G)$ désigne la connectivité par sommets, le plus petit nombre de sommets qu'il faut supprimer pour déconnecter le graphe (ou le réduire à un sommet),
- $\kappa'(G)$ désigne la connectivité par arêtes.

Cette conjecture est en réalité fausse. Le benchmark contient un contre-exemple connu, encodé au format **graph6** :

L~{?GKF@~o?~~{

Votre programme ne recevra pas ce contre-exemple pendant la recherche. Son objectif sera de le reconstruire automatiquement, ou de trouver un autre contre-exemple valide.

L'objectif du projet est donc de concevoir un programme capable de générer automatiquement des graphes, de calculer leurs invariants, puis de rechercher efficacement des graphes violant une conjecture donnée.

2 Objectif général

Chaque binôme doit concevoir la meilleure heuristique possible pour réfuter automatiquement des conjectures en théorie des graphes.

Pour chaque conjecture, le programme reçoit :

- une classe de graphes à respecter (connexe, sans griphe et planaire par exemple),
- une relation entre invariants,
- une limite de temps de recherche.

Le programme doit produire :

- un graphe candidat,
- son encodage au format **graph6**,
- les valeurs des invariants nécessaires,
- une preuve calculatoire que la conjecture est violée.

3 Outils autorisés

Tous les outils sont autorisés :

- IA génératives, comme ChatGPT, Claude, Gemini, Copilot ou Codex,
- agents IA,
- assistants de programmation,
- bibliothèques Python, comme NetworkX, igraph, numpy, pandas,
- solveurs exacts ou approchés,
- scripts générés automatiquement.

L'utilisation de l'IA n'est pas seulement autorisée. Elle est encouragée, à condition d'être comprise, maîtrisée et expliquée dans le rapport.

Il est interdit d'utiliser directement les contre-exemples connus du benchmark dans l'heuristique. Le programme soumis doit chercher les contre-exemples, non les recopier.

4 Invariants de graphes

Les conjectures manipulent notamment les invariants suivants :

$n, m, diam, rad, \delta, \Delta, avg, t, \omega, \gamma, \alpha, \tau, \mu, \kappa, \kappa'$.

Ces notations désignent respectivement :

Notation	Signification
n	nombre de sommets
m	nombre d'arêtes
$diam$	diamètre
rad	rayon
δ	degré minimum
Δ	degré maximum
avg	degré moyen
t	nombre de triangles
ω	taille d'une clique maximum
γ	nombre de domination
α	taille d'un ensemble indépendant maximum
τ	taille d'une couverture minimum par sommets
μ	taille d'un couplage maximum
κ	connectivité par sommets
κ'	connectivité par arêtes

Certaines conjectures sont restreintes à une classe de graphes :

$$G_{\text{connected}}, \quad G_{\text{planar}}, \quad G_{\text{bipartite}}, \quad G_{\text{tree}}, \quad G_{\text{claw-free}}.$$

5 Principe du score de violation

Pour une conjecture de la forme :

$$A(G) \leq f(B(G)),$$

on peut définir :

$$violation(G) = A(G) - f(B(G)).$$

Le graphe G est un contre-exemple si :

$$violation(G) > 0.$$

Pour une conjecture de la forme :

$$A(G) \geq f(B(G)),$$

on peut définir :

$$violation(G) = f(B(G)) - A(G).$$

L'objectif algorithmique est donc de maximiser la violation.

6 Partie 1 : première heuristique

La première partie du projet consiste à construire une heuristique simple, robuste et fonctionnelle.

Elle doit contenir au minimum :

1. une représentation des graphes,
2. un générateur de graphes initiaux,
3. une fonction de score,
4. des mutations locales,
5. une boucle de recherche,
6. un mécanisme de réparation si la classe du graphe n'est plus respectée.

6.1 Mutations possibles

Vous pouvez par exemple utiliser :

- ajout d’une arête,
- suppression d’une arête,
- ajout d’un sommet,
- suppression d’un sommet,
- subdivision d’une arête,
- ajout d’une feuille,
- ajout d’un chemin,
- ajout d’une clique,
- ajout de faux jumeaux,
- densification locale,
- transformation conservant la connexité,
- transformation conservant la planarité ou la bipartition.

6.2 Schéma algorithmique minimal

```
for conjecture in benchmark:
    population = generate_initial_graphs(conjecture)

    best_graph = None
    best_score = -inf

    while time_elapsed < 60:
        G = select_candidate(population)
        H = mutate(G)
        H = repair_if_needed(H, conjecture.graph_class)

        invariants = compute_invariants(H)
        score = violation_score(invariants, conjecture)

        if score > best_score:
            best_score = score
            best_graph = H

        if score > 0:
            return counterexample(H)

    return failure()
```

7 Partie 2 : architecture inspirée de FunSearch

La deuxième partie consiste à automatiser l’amélioration de la fonction de score.
L’idée est de ne plus se limiter à :

$$score(G) = violation(G).$$

On cherche une fonction plus informative :

$$F(G) = violation(G) + bonus(G) - penalty(G),$$

capable de guider la recherche vers des graphes prometteurs avant même qu’ils soient des contre-exemples.

7.1 Principe attendu

Votre architecture doit suivre l'idée suivante :

1. un LLM propose une fonction Python de score,
2. cette fonction est testée sur un sous-ensemble de conjectures,
3. les meilleures fonctions sont conservées,
4. le LLM reçoit les meilleures fonctions et propose des variantes,
5. le processus est répété.

7.2 Forme imposée de la fonction générée

```
def heuristic_score(G, invariants, conjecture):
    """
    G: Graphe NetworkX
    invariants: dictionnaire des invariants calculés
    conjecture: objet décrivant la conjecture
    retourne un score numérique à maximiser
    """
    violation = conjecture.violation(invariants)

    n = invariants.get("n", 0)
    m = invariants.get("m", 0)
    delta = invariants.get("delta", 0)
    Delta = invariants.get("Delta", 0)
    diam = invariants.get("diam", 0)
    gamma = invariants.get("gamma", 0)
    alpha = invariants.get("alpha", 0)
    tau = invariants.get("tau", 0)
    triangles = invariants.get("triangles", 0)

    return violation
```

Votre système doit chercher automatiquement à remplacer le dernier **return** par une fonction plus efficace.

7.3 Exemple de fonction candidate

```
def heuristic_score(G, invariants, conjecture):
    violation = conjecture.violation(invariants)

    n = invariants.get("n", 0)
    m = invariants.get("m", 0)
    diam = invariants.get("diam", 0)
    Delta = invariants.get("Delta", 0)
    triangles = invariants.get("triangles", 0)

    density = 0
    if n > 1:
        density = 2 * m / (n * (n - 1))

    return (
        10.0 * violation
        + 0.3 * diam
        + 0.2 * Delta
        + 0.1 * triangles
        - 0.05 * n
        - 0.2 * abs(density - 0.5)
```

)

Cette fonction n'est qu'un exemple. Vous devez essayer de faire mieux.

8 Évaluation des algorithmes

Chaque conjecture est évaluée avec une limite de temps de :

60 secondes.

Si votre programme trouve un contre-exemple en t_i secondes, le coût associé est :

$$c_i = t_i.$$

Si aucun contre-exemple n'est trouvé, le coût est :

$$c_i = 120.$$

Le score total est :

$$Score = \sum_i c_i.$$

Le meilleur binôme est celui qui obtient le score le plus faible.

9 Classement et notation

La note dépendra fortement du classement, mais pas uniquement.

Critère	Points
Classement final	8
Heuristique simple de la partie 1	3
Architecture FunSearch de la partie 2	4
Qualité du rapport	3
Qualité du dépôt GitHub	2
Total	20

9.1 Critères pour le classement

Le classement prendra en compte :

- le nombre de conjectures réfutées,
- le score total,
- le temps moyen de réfutation,
- la validité des contre-exemples produits,
- l'absence de solutions codées en dur.

10 Livrables

10.1 Dépôt GitHub

Chaque binôme doit fournir un dépôt GitHub contenant :

- le code source complet,
- un fichier `README.md`,
- un fichier `requirements.txt` ou équivalent,

- les scripts permettant de lancer les expériences,
- les résultats obtenus,
- le rapport PDF.

Structure recommandée :

```
graphbench-project/
|-- src/
|-- benchmark/
|-- experiments/
|-- results/
|-- README.md
|-- requirements.txt
|-- report.pdf
```

10.2 Rapport de 5 pages

Le rapport doit faire **10 pages maximum**, bibliographie comprise.

La page de garde doit obligatoirement contenir :

- le titre du projet,
- les noms et prénoms des deux étudiants,
- le groupe ou la formation,
- la date,
- le lien vers le dépôt GitHub,
- le nombre de conjectures réfutées,
- le score final.

10.3 Structure attendue du rapport

1. Introduction et objectif
2. Heuristique simple
3. Architecture FunSearch
4. Résultats expérimentaux
5. Discussion scientifique

La discussion doit répondre à des questions comme :

- quelles conjectures sont faciles à réfuter ?
- quels invariants semblent difficiles à manipuler ?
- quelles mutations sont efficaces ?
- l'architecture FunSearch améliore-t-elle réellement l'heuristique initiale ?
- l'IA a-t-elle produit des idées utiles ou seulement du code ?

11 Règles de validité

Un contre-exemple est accepté uniquement si :

1. le graphe appartient à la classe imposée,
2. les invariants sont correctement calculés,
3. l'inégalité est strictement violée,
4. le graphe est fourni au format `graph6`,

12 Interdictions

Sont interdits :

- coder directement les contre-exemples connus,

- utiliser l’identifiant d’une conjecture pour retourner une solution préenregistrée,
- modifier le benchmark d’évaluation,
- désactiver des conjectures difficiles,
- retourner un graphe qui ne respecte pas la classe demandée.

13 Conseils méthodologiques

Commencez simple. Une recherche locale bien codée peut être très efficace.

Vous pouvez ensuite améliorer progressivement :

- la génération initiale,
- les mutations,
- la réparation,
- le choix des graphes candidats,
- la fonction de score,
- la spécialisation par classe de graphes.

Il est souvent utile d’avoir plusieurs stratégies selon la classe :

- pour les graphes connexes, éviter les suppressions qui déconnectent,
- pour les arbres, travailler par ajout et suppression de feuilles,
- pour les graphes bipartis, maintenir explicitement la bipartition,
- pour les graphes planaires, tester la planarité après mutation,
- pour les graphes sans griffe, pénaliser fortement l’apparition de griffes induites.

Résumé

Vous devez construire, en binôme, un système capable de réfuter automatiquement des conjectures en théorie des graphes. Le projet comporte deux étapes. La première consiste à créer une heuristique simple mais fonctionnelle. La seconde consiste à mettre en place une architecture inspirée de FunSearch pour découvrir automatiquement de meilleures fonctions de score.

Votre objectif est double : obtenir le meilleur classement possible et produire une analyse scientifique claire de votre démarche.